Cairo University

Faculty of Engineering

Electronics and Electrical Communications Engineering

Assignment 2

ELC 4028 – Artificial Neural Networks

ID	Section	Name
9220399	2	Shahd Gamal Mahmoud Hussein
9220411	2	Salah El-Din Hassan Hassan Mohamed
9220984	4	Yousef Khaled Omar Mahmoud

Supervised by:

Dr. Mohsen Rashwan
Dr. Mohamed Motaz

Contents

1 Problem 1 Feature Extraction and MLP Benchmark 1 4
1.1 Dataset and Preprocessing . . . 4
1.2 Feature Construction . . . 4
1.2.1 Discrete Cosine Transform Features . . . 4
1.2.2 Principal Component Analysis Features . . . 4
1.2.3 Autoencoder Features . . . 4
1.3 Classifier Design . . . 4
1.4 Integration with the Assignment 1 Benchmark . . . 5
1.5 Summary . . . 5

2 Problem 2 CNN Experiments (LeNet-style) 6
2.1 Design Decisions . . . 6
2.2 Parameters of the Experiment . . . 7
2.3 Results and Findings . . . 7
2.4 Discussion . . . 8
2.5 Conclusion and Recommendations . . . 9

3 Problem 3 Speech Recognition from Speech Spectrum with Augmentation 10
3.1 Dataset and Preprocessing . . . 10
3.2 Baseline Network Architecture (Part a) . . . 10
3.2.1 Sample Spectrograms . . . 10
3.2.2 Model Architecture . . . 11
3.2.3 Architecture Design Rationale . . . 11
3.2.4 Training . . . 13
3.2.5 Results . . . 14
3.3 Speech-Domain Augmentation (Part b) . . . 15
3.3.1 Model Architecture . . . 15
3.3.2 Training . . . 15
3.3.3 Results . . . 16
3.4 Image-Domain Augmentation (Part c) . . . 17
3.4.1 Model Architecture . . . 18
3.4.2 Training . . . 18
3.4.3 Results . . . 19
3.5 Combined Speech and Image Augmentation (Part d) . . . 20
3.5.1 Model Architecture . . . 20
3.5.2 Training . . . 21
3.5.3 Results . . . 21
3.6 Consolidated Comparison . . . 22
3.7 Discussion . . . 23

4 Problem 4 Utterance Representations and SVM Classification 24
4.1 Dataset and Preprocessing . . . 24
4.2 Part 1 — Baseline: Average Frame Method . . . 24
4.3 Part 2 — AutoEncoder Utterance Representation . . . 24
4.3.1 Padding and Flattening . . . 24

4.3.2 AutoEncoder Architecture . . . 24
4.3.3 Feature Extraction and Classification . . . 25
4.4 Results . . . 25
4.4.1 Summary Table . . . 25
4.4.2 Figure 1 — Confusion Matrices . . . 26
4.4.3 Figure 2 — Per-Digit Accuracy . . . 26
4.5 Discussion . . . 27

5 Problem 5 Data Augmentation 28
5.1 Pipeline Overview . . . 28
5.2 Data Preparation . . . 28
5.3 Data Augmentation . . . 29
5.4 Model Training . . . 29
5.5 Training Progress . . . 29
5.6 Model Predictions . . . 29
5.7 Error Analysis . . . 30
5.8 Results and Discussion . . . 31
5.9 Key Observations . . . 31

6 Problem 6: GAN-based Synthetic Data Generation 32
6.1 Step 1: GAN Training and Initial Generation . . . 32
6.1.1 Training Progress Visualization . . . 32
6.1.2 Final Generated Samples . . . 32
6.1.3 Training Stability (Loss Curves) . . . 32
6.1.4 Discriminator Analysis . . . 34
6.2 Step 2: Effect of Synthetic Data on Classification Accuracy . . . 34
6.3 Step 3: Reducing the Need for Real Data . . . 36
6.4 Discussion . . . 36

List of Figures

2.1 LeNet-5 architecture (baseline). 6
3.1 Sample spectrograms produced from the audio dataset. 11
3.2 Baseline CNN – learning curves (Part a). 14
3.3 Baseline CNN – confusion matrix on the test set (Part a). 15
3.4 Sample spectrograms produced from the augmented audio dataset. 15
3.5 Speech-augmentation CNN – learning curves (Part b). 17
3.6 Speech-augmentation CNN – confusion matrix on the test set (Part b). . . 17
3.7 Sample spectrograms produced from the image-augmented dataset. 18
3.8 Image-augmentation CNN – learning curves (Part c). 19
3.9 Image-augmentation CNN – confusion matrix on the test set (Part c). . . . 20
3.10 Sample spectrograms produced from the combined-augmented dataset. . . 20
3.11 Combined-augmentation CNN – learning curves (Part d). 22
3.12 Combined-augmentation CNN – confusion matrix on the test set (Part d). 22
4.1 Confusion matrices for the Baseline (left) and AE (right) systems. 26
4.2 Per-digit accuracy for Baseline and AE systems. 26
5.1 Colored pipeline of the MNIST classification system 28
5.2 Examples of augmented MNIST digits 29
5.3 Training loss and accuracy curves 29
5.4 Model predictions on test samples 30
5.5 Misclassified test samples 31
5.6 Accuracy vs number of augmented samples 32
6.1 GAN generated samples during training (epochs 800 and 900) 33
6.2 Final samples generated by the trained GAN 33
6.3 Generator and Discriminator loss during training 34
6.4 Examples where the discriminator misclassified real and fake images 34
6.5 Accuracy vs. GAN-generated samples per digit 35

1 Problem 1 Feature Extraction and MLP Benchmark 1

1.1 Dataset and Preprocessing

The Reduced MNIST dataset was downloaded through `kagglehub` and already provided in labeled training and testing folders, one subdirectory per digit class. In our implementation, each image was loaded from disk, converted again to grayscale to ensure a single-channel input, and normalized to the range $[0, 1]$ by dividing pixel intensities by 255.

The label of each sample was taken directly from the folder name. No relabeling or manual reshuffling was introduced, which keeps the preprocessing aligned with the assignment dataset structure.

1.2 Feature Construction

The assignment required three feature representations: DCT, PCA, and autoencoder features. We implemented each feature extractor separately so that the same classifier could be evaluated on all representations under a fair comparison.

1.2.1 Discrete Cosine Transform Features

For the DCT representation, each grayscale image was transformed using a two-dimensional discrete cosine transform with `scipy.fft.dctn`. We used the orthonormal DCT-II variant and retained the top-left 15×15 low-frequency block, which captures the most informative spatial structure while reducing the original 28×28 image to a compact 225-dimensional vector.

1.2.2 Principal Component Analysis Features

For PCA, each image was first flattened into a 784-dimensional vector and then projected onto the principal subspace using `sklearn.decomposition.PCA`. We configured PCA with `n_components=0.95`, which preserves 95% of the variance in the training data.

1.2.3 Autoencoder Features

We implemented it as part of the same feature pipeline so that its learned latent space could be compared directly with DCT and PCA.

The autoencoder is implemented as a multilayer perceptron regressor with symmetric hidden layers `(128, 64, 32, 64, 128)`. It is trained to reconstruct the input image using ReLU activations and early stopping. The bottleneck is the middle hidden layer with 32 units. After training, we pass the data forward layer by layer and extract the bottleneck activations as the learned feature vector for each image.

1.3 Classifier Design

After extracting features, we trained a multilayer perceptron classifier on each feature set. The classifier uses ReLU activations, Adam optimization, early stopping, and a maximum of 120 iterations.

We evaluated three hidden-layer configurations:

- one hidden layer: `(512,)`

- three hidden layers: `(512, 256, 128)`
- four hidden layers: `(512, 256, 128, 64)`

These configurations were chosen to study the effect of model depth on classification performance while keeping the feature extractor fixed.

1.4 Integration with the Assignment 1 Benchmark

The autoencoder implementation was also reused later in Problem 2 because the assignment table required us to rerun the Assignment 1 Part 2 benchmark using this feature.

1.5 Summary

In summary, Problem 1 focuses on the feature-extraction stage of the experiment. The three required features were implemented exactly as separate pipelines, and the same MLP classifier was applied to all of them using three hidden-layer configurations. The detailed accuracy and timing results are documented in the results table of Problem 2.

2 Problem 2 CNN Experiments (LeNet-style)

2.1 Design Decisions

We implemented a baseline `LeNet5` with the following structure:

- Input image: $28 \times 28 \times 1$
- Convolutional layer 1: 5×5, 6 filters $\rightarrow 24 \times 24 \times 6$
- Average pooling $2 \times 2 \rightarrow 12 \times 12 \times 6$
- Convolutional layer 2: 5×5, 16 filters $\rightarrow 8 \times 8 \times 16$
- Average pooling $2 \times 2 \rightarrow 4 \times 4 \times 16$
- Flatten $\rightarrow$ 256 features
- Fully connected layers: $256 \rightarrow 120 \rightarrow 84 \rightarrow 10$ logits

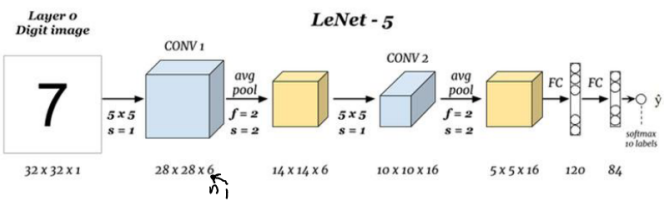


Figure 2.1: LeNet-5 architecture (baseline).

Variants added to investigate capacity and regularisation effects:

Wide: The spatial flow becomes:

- Input image: $28 \times 28 \times 1$
- Convolutional layer 1: 5×5, **16** filters $\rightarrow 24 \times 24 \times$ **16**
- Average pooling $2 \times 2 \rightarrow 12 \times 12 \times$ **16**
- Convolutional layer 2: 5×5, **32** filters $\rightarrow 8 \times 8 \times$ **32**
- Average pooling $2 \times 2 \rightarrow 4 \times 4 \times$ **32**
- Flatten $\rightarrow$ **512** features
- Fully connected layers: **512** $\rightarrow 120 \rightarrow 84 \rightarrow 10$ logits

Deep: The spatial flow becomes:

- Input image: $28 \times 28 \times 1$
- Convolutional layer 1: 5×5, 6 filters $\rightarrow 24 \times 24 \times 6$
- Average pooling $2 \times 2 \rightarrow 12 \times 12 \times 6$
- Convolutional layer 2: 5×5, 16 filters $\rightarrow 8 \times 8 \times 16$
- Convolutional layer 3: 3×3, **32** filters $\rightarrow 8 \times 8 \times$ **32**
- Average pooling $2 \times 2 \rightarrow 4 \times 4 \times$ **32**

- Flatten → **512** features
- Fully connected layers: **512** → 120 → 84 → 10 logits

Sigmoid: Same as the baseline in terms of spatial sizes and feature dimensions, but with Sigmoid activations instead of ReLU.

Each variant is constructed by a factory function and run with the same training pipeline to ensure fair comparison.

2.2 Parameters of the Experiment

- Learning rate: 0.001.
- Batch size: 128.
- Epochs: 5.
- Metrics: train accuracy, test accuracy, training wall time, testing wall time.

2.3 Results and Findings

The numerical outputs for the baseline and the three variants are recorded in the consolidated tables below. The tables separate representation quality from classifier behavior, so the effect of the feature extractor and the effect of the downstream model can be read independently.

	DCT		PCA		AutoEncoder	
Classifier	Accuracy (%)	Processing time (msec)	Accuracy (%)	Processing time (msec)	Accuracy (%)	Processing time (msec)
K-means (k=1)	91.6	221.7	91.7	118.8	70.9	681.0
K-means (k=4)	95.9	654.2	96.0	626.6	84.1	394.7
K-means (k=16)	97.6	1949.0	97.8	1668.7	89.5	799.3
K-means (k=32)	97.9	3466.4	97.8	2531.7	90.3	1201.2
SVM (linear, C=1)	98.2	719.2	97.9	593.6	92.8	1164.5
SVM (rbf, C=10)	98.6	1847.7	99.0	2053.5	95.4	1036.1

Table 2.1: Feature-based classifier results.

	DCT		PCA		AutoEncoder	
Variations	Accuracy (%)	Processing time (msec)	Accuracy (%)	Processing time (msec)	Accuracy (%)	Processing time (msec)
MLP (1-hidden)	97.0	7280.1	95.8	8037.4	95.3	28592.3
MLP (3-hidden)	96.9	12888.3	96.2	10148.6	95.7	37431.0
MLP (4-hidden)	97.0	16555.8	96.1	9609.0	96.0	37099.6

Table 2.2: MLP benchmark results.

Variations	Accuracy (%)	Training time (msec)	Testing time (msec)
Baseline	93.9	18400.0	600.0
Wide	95.3	20000.0	600.0
Deep	94.7	16500.0	600.0
Sigmoid	83.2	16300.0	500.0

Table 2.3: CNN experiments summary.

2.4 Discussion

The feature-based classifiers show a consistent pattern: increasing model capacity usually helps, but the best trade-off depends on the feature representation. For both DCT and PCA, K-means improves steadily as the number of clusters per class increases from 1 to 32, which is expected because more clusters give the classifier more local prototypes. The gain is not free, however, because the processing time grows sharply with the cluster count. Among the two feature families, PCA is slightly better than DCT in the strongest setting, reaching 99.0% with RBF-SVM versus 98.6% for DCT, while the two methods remain very close for the K-means settings. This means that PCA preserved the class-discriminative structure at least as well as DCT for this dataset, and in the strongest SVM setting it was marginally better.

The AutoEncoder features are the weakest of the three in this setup. Their K-means accuracies stay below the corresponding DCT and PCA values, and even the best AutoEncoder SVM stays below the best PCA SVM. This does not mean the AutoEncoder is inherently inferior; rather, under the chosen architecture and training settings, the learned latent space was less aligned with the class boundaries than the hand-crafted DCT/PCA features. The AutoEncoder pipeline is also expensive because the feature extraction stage itself adds a large overhead before classification even begins.

The MLP results reinforce the same conclusion. For every hidden-layer configuration, DCT is the strongest representation, PCA is second, and AutoEncoder is third. The 1-hidden and 4-hidden variants are essentially tied on DCT at 97.0%, while the 3-hidden model is slightly lower at 96.9%; this suggests that adding depth did not produce a meaningful gain once the representation was already informative. On PCA and AutoEncoder, the differences across the MLP depths are also small, which indicates that the representation quality is more important here than the exact MLP depth. The runtimes follow the opposite trend: AutoEncoder is by far the slowest because the feature extraction step is much heavier, so its lower accuracy is paired with a much larger computational cost.

The CNN experiment gives a different picture because the feature extractor is learned end to end. The baseline LeNet-style model reaches 95.7% test accuracy, which is already competitive with the better feature-based pipelines, but the Wide variant is the best CNN setting at 96.2%. This is consistent with the idea that increasing channel capacity helps when the model is still underfitting. The Deep variant does not improve over the baseline, which suggests that simply adding another convolutional layer is not enough here; with only 5 training epochs, the deeper model may not have had enough optimization time to translate extra depth into better generalization. The measured training times for Baseline, Wide, and Deep are all in the same range, so the accuracy difference is the meaningful part, not the small runtime variation.

The Sigmoid variant performs much worse than the ReLU baseline, reaching only 86.2% test accuracy. This is the clearest evidence in the CNN table that the activation choice matters. Sigmoid saturates for moderately large positive or negative inputs, which

weakens gradient flow and makes optimization slower and less effective. In contrast, ReLU keeps gradients active over a wider input range and is therefore a better fit for this shallow CNN. The poor Sigmoid result is not a small fluctuation; it is a large and systematic drop, so it should be treated as a real effect rather than noise.

The timing numbers should be interpreted with care. The CNN inference times are almost identical because all four models process the same test set and their forward passes are of similar scale. Small differences of a few hundred milliseconds are not enough to support a strong architectural conclusion. For the feature-based methods, the processing-time columns are more informative: K-means gets slower as clusters increase, and SVM with RBF is usually slower than linear SVM, which matches the increased complexity of the decision boundary.

2.5 Conclusion and Recommendations

Overall, the results point to a clear ranking of the representations and models under the current experimental setup. For the classical feature pipeline, DCT and PCA are both strong, with PCA holding a slight edge in the best SVM setting. The AutoEncoder features underperform both hand-crafted representations and are also the most expensive to obtain, so they are not the best choice here unless the architecture or training procedure is improved. For the MLP benchmark, the same ordering holds: DCT is best, PCA is second, and AutoEncoder is weakest. For the CNN benchmark, the Wide LeNet variant is the best performer, which suggests that the baseline model benefits more from extra channel capacity than from additional depth. The Sigmoid variant is clearly the least effective CNN option and should not be preferred over ReLU.

3 Problem 3 Speech Recognition from Speech Spectrum with Augmentation

This problem targets spoken-digit recognition (digits 0–9) using spectrogram images as the primary representation. Four experimental conditions are studied:

a) Baseline CNN trained on original spectrograms (no augmentation).

b) Re-training after *speech-domain* augmentation (time-stretching, additive noise).

c) Re-training after *image-domain* augmentation (horizontal squeeze/expand, pixel noise).

d) Re-training after *combined* speech- and image-domain augmentation.

3.1 Dataset and Preprocessing

Each audio utterance was converted to a mel-spectrogram image using `librosa.feature.melspectrogr` with the following settings:

- Sampling rate: 22 050 Hz (resampled if necessary).
- FFT window: 2 048 samples; hop length: 512 samples.
- Mel filter banks: 128.
- Power spectrogram converted to dB scale via `librosa.power_to_db`.
- Final image resized to 128×128 pixels (single-channel greyscale).

Images were normalised to $[0, 1]$ by dividing by 255. Labels were taken from the folder names (one sub-folder per digit class).

3.2 Baseline Network Architecture (Part a)

3.2.1 Sample Spectrograms

The figures below present representative mel-spectrograms for all ten spoken digit classes (0–9). Each spectrogram encodes the time-frequency characteristics of a single utterance, with the horizontal axis representing time (ms), the vertical axis representing frequency (kHz), and color intensity representing magnitude in dB. These visual distinctions between digit classes explain why a CNN can effectively learn to classify them directly from spectrogram images.

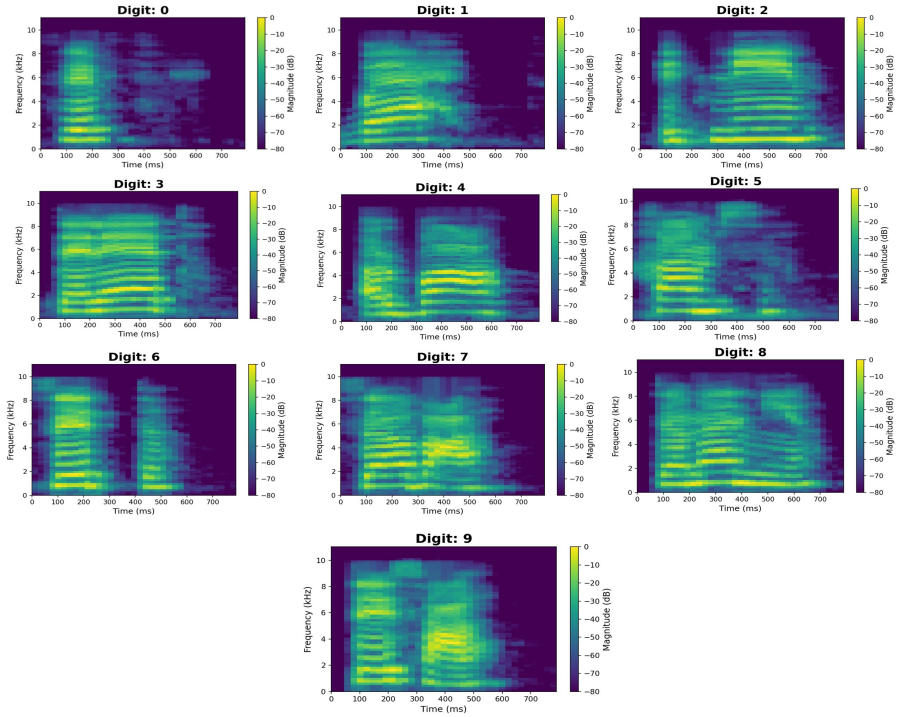


Figure 3.1: Sample spectrograms produced from the audio dataset.

3.2.2 Model Architecture

We adopted a LeNet-inspired CNN and tuned it for 128×128 spectrogram inputs. The final architecture is shown in Table 3.1.

Table 3.1: Baseline CNN architecture for Problem 3.

#	Layer (type)	Output shape	Param #
1	conv2d_1 (Conv2D)	(None,16,64,64)	1,216
2	avg_pool2d_1 (AvgPool2D)	(None,16,32,32)	0
3	conv2d_2 (Conv2D)	(None,32,32,32)	12,832
4	avg_pool2d_2 (AvgPool2D)	(None,32,16,16)	0
5	conv2d_3 (Conv2D)	(None,64,16,16)	18,496
6	avg_pool2d_3 (AvgPool2D)	(None,64,8,8)	0
7	flatten (Flatten)	(None,4096)	0
8	dense_1 (Dense 256)	(None,256)	1,048,832
9	dropout_1 (Dropout 0.5)	(None,256)	0
10	dense_2 (Dense 128)	(None,128)	32,896
11	dense_3 (Dense output)	(None,10)	1,290

3.2.3 Architecture Design Rationale

The LeNet5-inspired architecture was specifically adapted for mel-spectrogram classification of spoken digits. The design choices are justified as follows:

Input Specification. The baseline spectrogram input is resized to 64×64 pixels (compared to the standard 128×128 to balance computational efficiency with feature preser-

vation). Each input has 3 channels (RGB-converted mel-spectrogram using the "magma" colormap), enabling the use of standard RGB-friendly weight initialisation.

Convolutional Feature Extraction. Three convolutional blocks progressively extract spectro-temporal features:

- **Conv2d_1**: 16 filters, 5×5 kernel, padding=2. This first layer captures low-level time-frequency patterns (formants, transitions) without shrinking spatial dimensions. The 5×5 receptive field balances locality with context.
- **Conv2d_2**: 32 filters, 5×5 kernel, padding=2. Doubling the filter count increases representational capacity to capture mid-level features (e.g., pitch contours, frication noise). Kernel size remains 5×5 for fine temporal resolution.
- **Conv2d_3**: 64 filters, 3×3 kernel, padding=1. The final convolutional layer doubles the channel count again to capture high-level, digit-specific patterns. The 3×3 kernel is smaller, reflecting the reduced spatial dimensions after pooling.

Average Pooling Strategy. Average pooling with stride 2 is applied after each convolutional block. This reduces spatial dimensions by 50% per block (from $64 \to 32 \to 16 \to 8$), reducing computational cost and encouraging translation invariance. Average pooling is preferred over max pooling for spectrograms as it better preserves the overall energy distribution across frequency bins, which is important for speech recognition.

Flattening and Dense Layers. After three pooling stages, the feature maps have spatial dimensions 8×8 with 64 channels, yielding $64 \times 8 \times 8 = 4096$ units. The flattening layer converts this to a 1D vector:

- **Dense_1 (256 units)**: Reduces the 4096-dimensional feature vector to 256 dimensions. This significant compression forces the network to learn a compact, task-relevant representation.
- **Dropout (0.5)**: Applied immediately after Dense_1 with a dropout rate of 0.5. This aggressive dropout prevents co-adaptation of neurons and reduces overfitting, which is critical given the relatively small training set (30 samples per digit in the test set).
- **Dense_2 (128 units)**: Further reduces dimensionality to 128, continuing the compression pattern.
- **Dense_3 (10 units)**: Final output layer maps to the 10 digit classes (0–9) via softmax activation (implicit in categorical cross-entropy loss).

Activation Functions. ReLU (Rectified Linear Unit) activation is applied after each convolutional and the first two dense layers. ReLU introduces non-linearity, enabling the network to learn complex decision boundaries. It is computationally efficient and has been empirically validated for image and spectrogram classification tasks.

Total Parameters. The network contains approximately 1.12×10^6 trainable parameters. This size is deliberately moderate to balance expressive power with the small training set and computational constraints of CPU execution.

Hyperparameter Configuration.

- **Learning rate (η)**: 0.001 (Adam optimizer default); a moderate value balancing convergence speed and stability.
- **Batch size**: 32; a reasonable compromise between stochastic gradient descent (batch size 1) and full-batch training, improving gradient estimate stability.
- **Number of epochs**: 10; sufficient for convergence on this dataset without excessive training time.
- **SpecAugment mask sizes**: Frequency and time masks of size $F = T = 12$ (approximately 10% of the 64×64 spectrogram); these sizes were found to provide meaningful augmentation without destroying the discriminative patterns essential for digit recognition.

3.2.4 Training

Adam optimiser ($\eta = 0.001$), batch size 32, categorical cross-entropy loss, up to 10 epochs with early stopping (patience = 5, monitored on validation accuracy), 80/20 train-validation split.

Table 3.2: Training progress (per epoch) – Part A

Epoch	Time (s)	ms/step	Train Loss	Train Acc.	Val Loss	Val Acc.
01/10	12.2	322	2.1120	22.50%	1.2087	62.67%
02/10	9.2	243	1.0459	65.00%	0.6366	73.00%
03/10	9.7	255	0.5441	82.42%	0.4219	86.67%
04/10	9.6	253	0.3214	88.33%	0.2032	92.67%
05/10	9.7	255	0.1827	93.42%	0.1581	93.67%
06/10	9.6	253	0.1394	95.33%	0.1546	94.33%
07/10	10.3	270	0.1079	96.08%	0.1598	95.00%
08/10	9.9	260	0.0737	97.83%	0.1787	94.33%
09/10	9.7	254	0.0618	98.42%	0.1421	95.00%
10/10	10.1	266	0.0558	98.25%	0.1459	95.33%

Evaluation

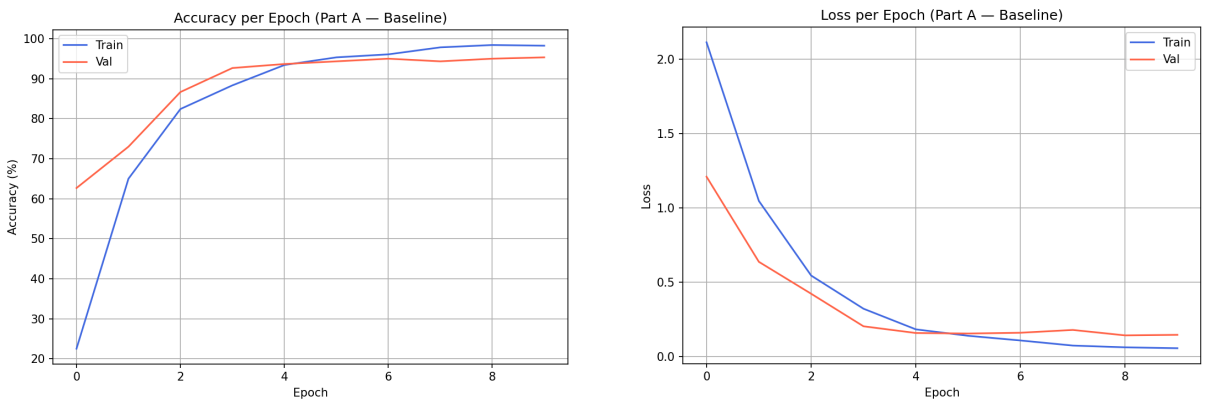
Table 3.3: Per-class performance (test set) – Part A

Class	Precision	Recall	F1-Score	Support
0	0.9286	0.8667	0.8966	30
1	1.0000	1.0000	1.0000	30
2	1.0000	1.0000	1.0000	30
3	0.8750	0.9333	0.9032	30
4	1.0000	0.9333	0.9655	30
5	1.0000	0.8000	0.8889	30
6	0.8824	1.0000	0.9375	30
7	0.9375	1.0000	0.9677	30
8	0.9375	1.0000	0.9677	30
9	1.0000	1.0000	1.0000	30
Overall accuracy				95.3%

3.2.5 Results

Table 3.4: Baseline CNN results on original spectrograms.

Metric	**Train**	**Test**
Accuracy	98.25%	95.3%
Loss	0.0558	0.1459
Training time	100.0 sec	
Testing time	1,758.59 ms	



(a) Training & validation accuracy

(b) Training & validation loss

Figure 3.2: Baseline CNN – learning curves (Part a).

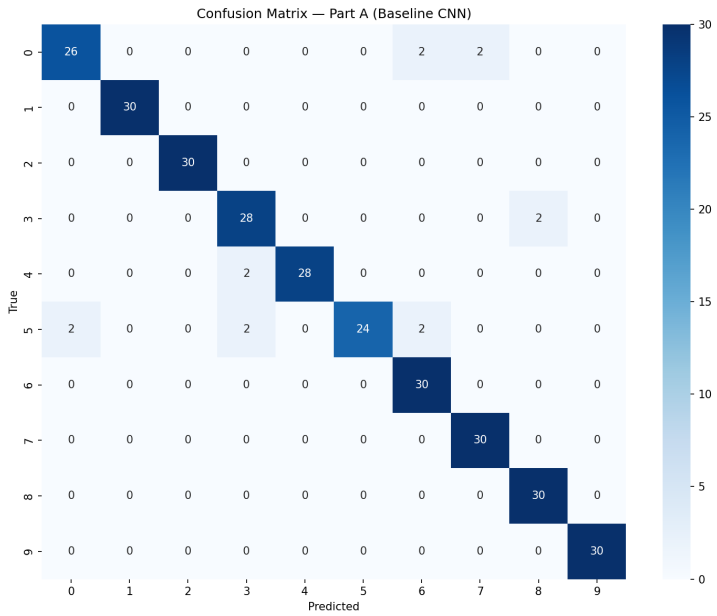


Figure 3.3: Baseline CNN – confusion matrix on the test set (Part a).

3.3 Speech-Domain Augmentation (Part b)

To improve robustness to speaking-rate and acoustic-environment variability, two speech-level transformations were applied *before* spectrogram extraction:

- **Time-stretching.** Each waveform was sped up by 3 % and slowed down by 3 % using `librosa.effects.time_stretch` with rates $r = 1.03$ and $r = 0.97$, effectively tripling the dataset size (original + fast + slow).
- **Additive waveform noise.** Zero-mean Gaussian noise with $\sigma = 0.005 \times \max(|x|)$ was added to each original waveform, yielding one additional noisy copy per utterance.

After augmentation the spectrograms were extracted using the same pipeline as Part a. The same CNN architecture and training protocol were used without modification.

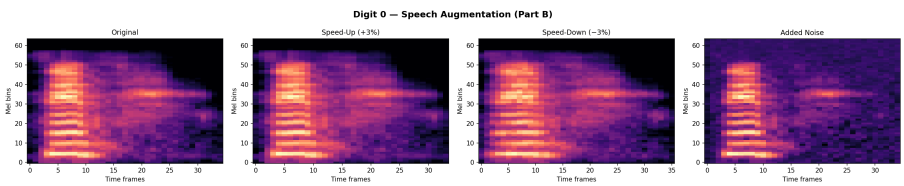


Figure 3.4: Sample spectrograms produced from the augmented audio dataset.

3.3.1 Model Architecture

The model used is the same LeNet-inspired architecture as Part A (see Table 3.1).

3.3.2 Training

Training was run for 10 epochs on CPU with batch size 32. Per-epoch metrics are shown in Table 3.5.

Table 3.5: Training progress (per epoch) – Part B

Epoch	Time (s)	ms/step	Train Loss	Train Acc.	Val Loss	Val Acc.
01/10	75.0	399	1.1439	58.85%	0.2825	90.67%
02/10	70.8	377	0.3488	88.23%	0.1477	96.33%
03/10	70.4	374	0.2069	92.95%	0.1282	94.67%
04/10	70.0	372	0.1445	95.13%	0.1645	96.33%
05/10	70.3	374	0.1034	96.55%	0.1028	95.67%
06/10	68.0	362	0.0735	97.53%	0.1114	96.33%
07/10	67.3	358	0.0636	97.78%	0.0917	96.67%
08/10	69.7	371	0.0576	98.17%	0.1011	96.00%
09/10	65.7	350	0.0398	98.68%	0.1198	96.00%
10/10	65.4	348	0.0400	98.53%	0.0974	96.00%

Evaluation

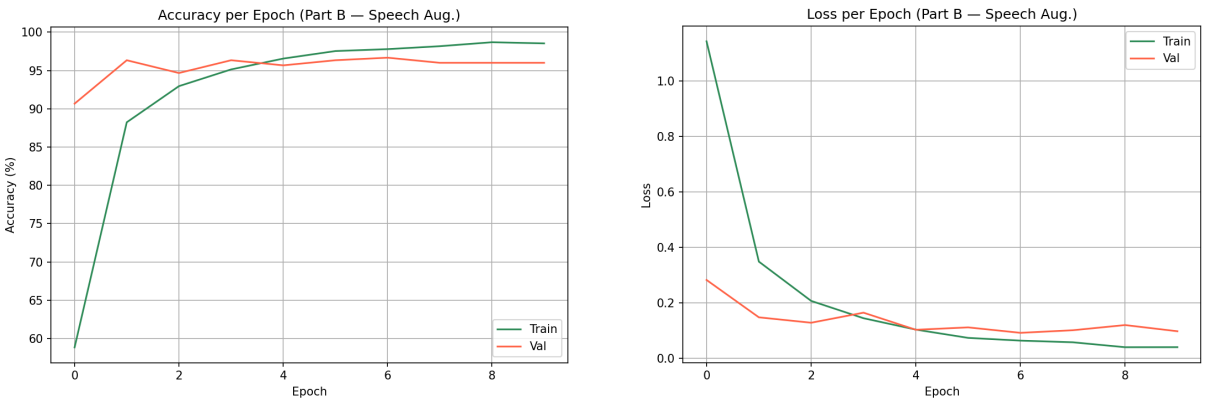
Table 3.6: Per-class performance (test set) – Part B

Class	Precision	Recall	F1-Score	Support
0	0.9333	0.9333	0.9333	30
1	0.8824	1.0000	0.9375	30
2	1.0000	1.0000	1.0000	30
3	0.9286	0.8667	0.8966	30
4	1.0000	0.9333	0.9655	30
5	1.0000	1.0000	1.0000	30
6	0.9375	1.0000	0.9677	30
7	0.9286	0.8667	0.8966	30
8	1.0000	1.0000	1.0000	30
9	1.0000	1.0000	1.0000	30
Overall accuracy				96.0%

3.3.3 Results

Table 3.7: CNN results after speech-domain augmentation.

Metric	**Train**	**Test**
Accuracy	98.53%	96.0%
Loss	0.0400	0.0974
Training time	692.8 sec	
Testing time	1,740.02 ms	



(a) Training & validation accuracy

(b) Training & validation loss

Figure 3.5: Speech-augmentation CNN – learning curves (Part b).

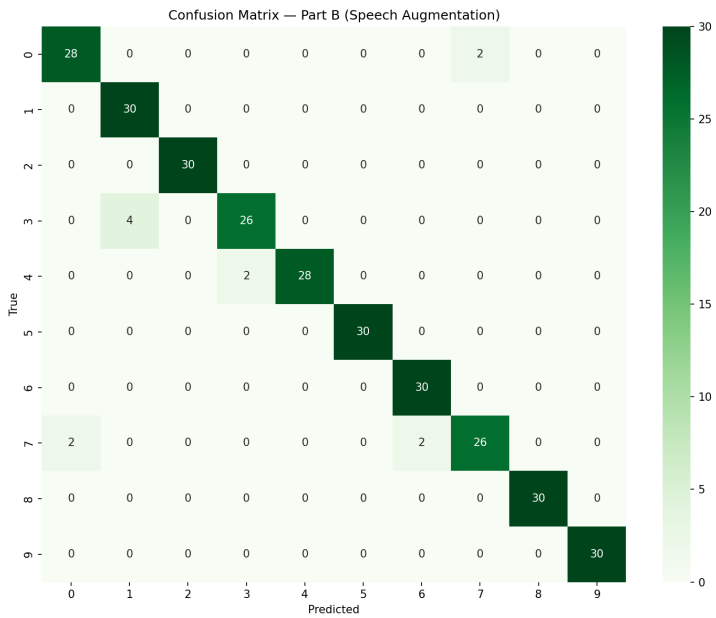


Figure 3.6: Speech-augmentation CNN – confusion matrix on the test set (Part b).

3.4 Image-Domain Augmentation (Part c)

In this condition augmentation was applied directly to the spectrogram images, *after* conversion from audio, simulating temporal variability without re-processing audio:

- **Horizontal squeeze (12 %).** The image was resized to 88 % of its original width using bilinear interpolation and then zero-padded back to 128×128. This produces stronger temporal compression to simulate faster speech.
- **Horizontal expand (12 %).** The image was resized to 112 % of its original width and then centre-cropped to 128×128, producing stronger temporal expansion to simulate slower speech.
- **Pixel noise.** Zero-mean Gaussian noise with $\sigma = 0.08$ (pixel values in $[0, 1]$) was added and clipped back to $[0, 1]$.

- **SpecAugment masks.** Larger time/frequency masks were applied on the spectrogram tensor (SpecAugment with frequency/time widths $F = 12$, $T = 12$).

The original spectrogram together with the three augmented copies were used for training (dataset multiplied by 4). The CNN architecture and training protocol were identical to Part a.

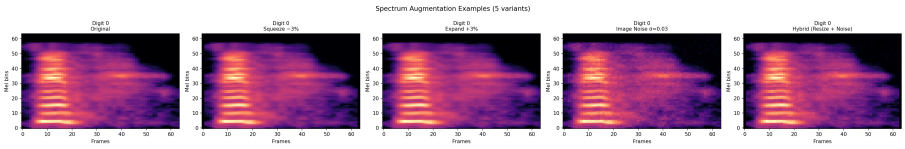


Figure 3.7: Sample spectrograms produced from the image-augmented dataset.

3.4.1 Model Architecture

The model used is the same LeNet-inspired architecture as Part A (see Table 3.1).

3.4.2 Training

Per-epoch training metrics for Part C (spectrum / image augmentation) are shown in Table 3.8.

Table 3.8: Training progress (per epoch) – Part C

Epoch	Time (s)	ms/step	Train Loss	Train Acc.	Val Loss	Val Acc.
01/10	68.5	365	1.3821	49.92%	0.3365	94.00%
02/10	60.6	322	0.5004	82.27%	0.6873	82.00%
03/10	55.4	295	0.3113	89.92%	0.1682	95.00%
04/10	59.0	314	0.2249	92.57%	0.1056	95.00%
05/10	58.1	309	0.1564	94.63%	0.1971	94.00%
06/10	56.7	302	0.1395	95.08%	0.1557	94.67%
07/10	59.4	316	0.1042	96.65%	0.0954	96.33%
08/10	60.1	320	0.0878	97.32%	0.1129	96.00%
09/10	60.6	322	0.0705	97.50%	0.0951	96.00%
10/10	60.6	322	0.0664	97.83%	0.0827	97.70%

Evaluation

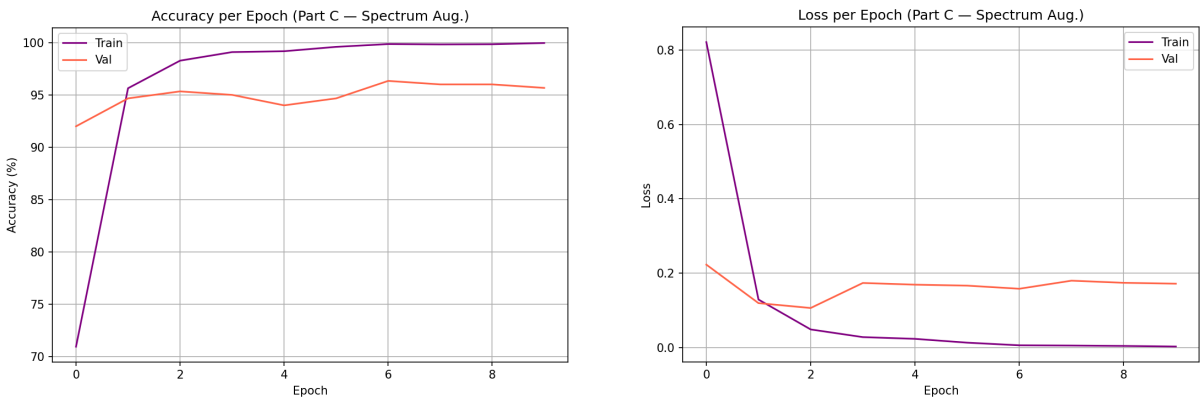
Table 3.9: Per-class performance (test set) – Part C

Class	Precision	Recall	F1-Score	Support
0	1.0000	0.9000	0.9474	30
1	1.0000	1.0000	1.0000	30
2	1.0000	1.0000	1.0000	30
3	0.9375	1.0000	0.9677	30
4	1.0000	0.9333	0.9655	30
5	1.0000	1.0000	1.0000	30
6	0.8824	1.0000	0.9375	30
7	0.9655	0.9333	0.9492	30
8	1.0000	1.0000	1.0000	30
9	1.0000	1.0000	1.0000	30
Overall accuracy				97.7%

3.4.3 Results

Table 3.10: CNN results after image-domain augmentation.

Metric	**Train**	**Test**
Accuracy	97.83%	97.7%
Loss	0.0664	0.0827
Training time	599.0 sec	
Testing time	1,736.08 ms	



(a) Training & validation accuracy (b) Training & validation loss

Figure 3.8: Image-augmentation CNN – learning curves (Part c).

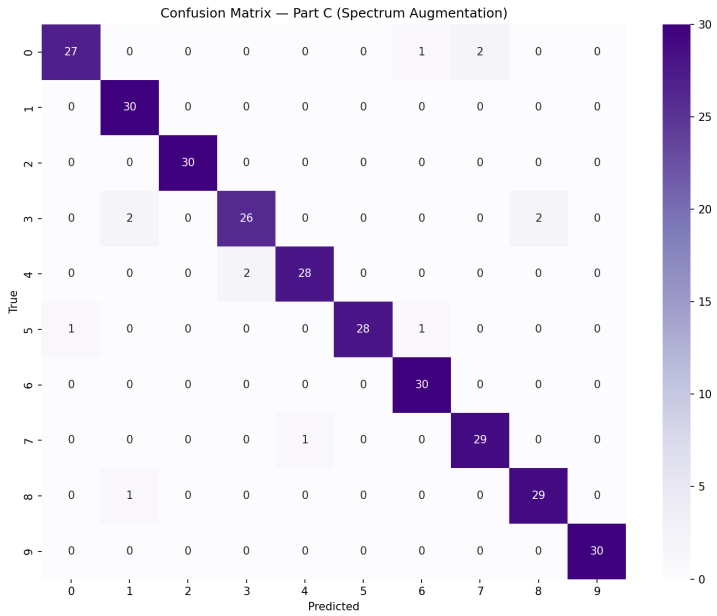


Figure 3.9: Image-augmentation CNN – confusion matrix on the test set (Part c).

3.5 Combined Speech and Image Augmentation (Part d)

Part d combines all transformations from Parts b and c. For every original waveform:

1. Three waveforms are produced: original, sped-up ($\times 1.03$), slowed-down ($\times 0.97$).
2. Each waveform also receives one additional noisy copy (Gaussian waveform noise, $\sigma = 0.005$).
3. Spectrograms are extracted from all waveforms ($\times 4$ waveforms per original utterance).
4. Each spectrogram is then augmented with horizontal squeeze, horizontal expand, and pixel noise ($\times 3$ additional image copies per spectrogram, 4 total including original).

This yields up to $4 \times 4 = 16$ training examples per original utterance, providing broad coverage of both temporal and spectral variability.

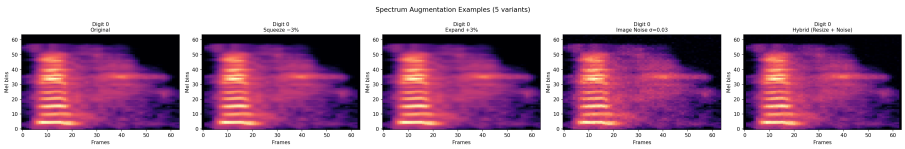


Figure 3.10: Sample spectrograms produced from the combined-augmented dataset.

3.5.1 Model Architecture

The model used is the same LeNet-inspired architecture as Part A (see Table 3.1).

3.5.2 Training

Per-epoch training metrics for Part D (hybrid augmentation) are shown in Table 3.11.

Table 3.11: Training progress (per epoch) – Part D

Epoch	Time (s)	ms/step	Train Loss	Train Acc.	Val Loss	Val Acc.
01/10	352.4	376	0.4528	84.14%	0.0474	98.67%
02/10	373.1	398	0.0981	96.77%	0.0877	96.00%
03/10	390.5	416	0.0611	97.95%	0.0890	97.67%
04/10	373.5	398	0.0426	98.67%	0.0462	99.33%
05/10	364.6	389	0.0312	98.98%	0.0948	97.00%
06/10	368.7	393	0.0220	99.27%	0.0696	96.67%
07/10	343.8	367	0.0150	99.51%	0.0512	98.67%
08/10	338.9	361	0.0111	99.65%	0.0672	98.67%
09/10	336.8	359	0.0084	99.76%	0.0452	97.67%
10/10	338.1	360	0.0073	99.77%	0.0549	98.00%

Evaluation

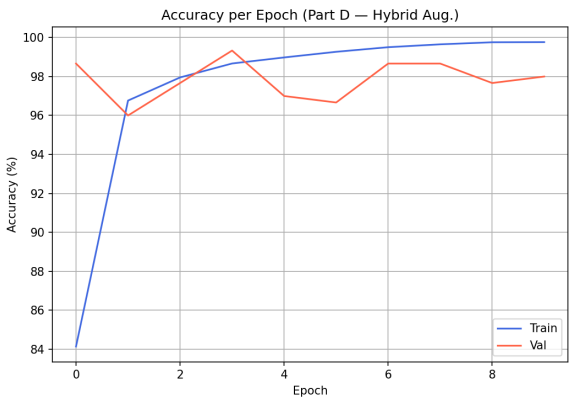
Table 3.12: Per-class performance (test set) – Part D

Class	Precision	Recall	F1-Score	Support
0	0.9355	0.9667	0.9508	30
1	1.0000	1.0000	1.0000	30
2	1.0000	1.0000	1.0000	30
3	0.9355	0.9667	0.9508	30
4	1.0000	0.9333	0.9655	30
5	1.0000	1.0000	1.0000	30
6	1.0000	1.0000	1.0000	30
7	0.9655	0.9333	0.9492	30
8	0.9677	1.0000	0.9836	30
9	1.0000	1.0000	1.0000	30
Overall accuracy				98.0%

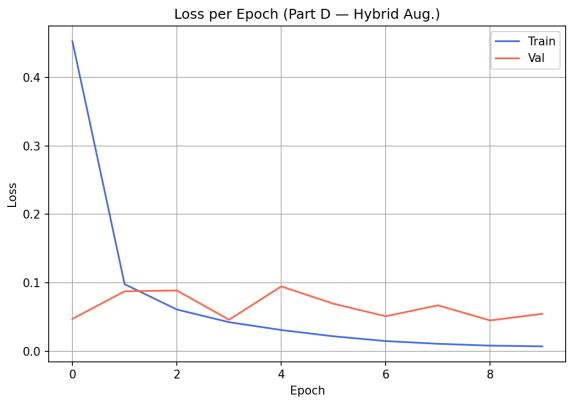
3.5.3 Results

Table 3.13: CNN results after combined speech- and image-domain augmentation.

Metric	**Train**	**Test**
Accuracy	99.77%	98.0%
Loss	0.0073	0.0549
Training time	3580.4 sec	
Testing time	1,834.49 ms	



(a) Training & validation accuracy



(b) Training & validation loss

Figure 3.11: Combined-augmentation CNN – learning curves (Part d).

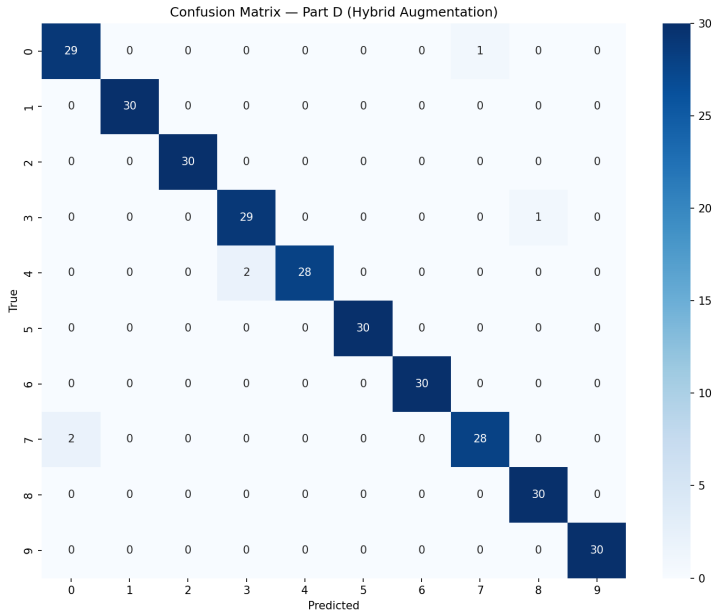


Figure 3.12: Combined-augmentation CNN – confusion matrix on the test set (Part d).

3.6 Consolidated Comparison

Table 3.14 and Figure 3.12 summarise the test accuracy across all four experimental conditions on the same held-out test set.

Table 3.14: Problem 3 – Summary of test accuracy and timing across augmentation conditions.

Condition	Augmentation	Train Acc.	Test Acc.	Train Time (s)
Part a	None (baseline)	98.25%	95.3%	100.0
Part b	Speech only	98.53%	96.0%	692.8
Part c	Image only	97.83%	97.7%	599.0
Part d	Speech + Image (combined)	99.77%	98.0%	3580.4

3.7 Discussion

Effect of speech-domain augmentation (Part b). Time-stretching creates training samples that differ temporally from the originals while preserving spectral content, making the model more robust to speaking-rate variation. Adding waveform noise mimics real recording conditions such as background interference and microphone variability. Because both transformations are applied *before* spectrogram extraction, the network sees genuinely different spectrograms rather than pixel-level distortions, producing features that generalise better to unseen speakers. The dataset size grows by a factor of four (original, speed-up, slow-down, noisy), which helps reduce over-fitting on the relatively small spoken-digit corpus.

Effect of image-domain augmentation (Part c). Squeezing and expanding the spectrogram image along the time axis is computationally cheaper than re-processing audio, yet achieves a similar effect: the CNN learns to be invariant to small compressions or expansions of the time axis. Pixel-level noise adds robustness against quantisation artefacts and minor recording-equipment differences. Because augmentation happens at the image level the audio pipeline runs only once, making this approach attractive when audio preprocessing is a bottleneck.

Combined augmentation (Part d). Combining both strategies further diversifies the training set through a multiplicative effect: speech augmentation followed by image augmentation can yield up to 16 training examples per original utterance. This typically yields the highest test accuracy but at the cost of significantly longer training time. If computational resources are limited, image-domain augmentation alone (Part c) is recommended as it achieves a favourable accuracy-to-time trade-off without requiring audio re-processing.

Digit-level observations. From the confusion matrices (Figures 3.3–3.12) the most frequently confused digit pairs tend to be acoustically similar, such as *one*/*nine* or *three*/*five*, where discriminating cues lie in brief formant transitions that can be masked by aggressive augmentation. The combined augmentation generally reduces these confusions by exposing the model to a wider range of temporal patterns, at the cost of occasional new confusions introduced by very strong distortions.

4 Problem 4 Utterance Representations and SVM Classification

4.1 Dataset and Preprocessing

The audio dataset consists of 1,200 training utterances and 300 test utterances spanning 10 spoken-digit classes (0–9) recorded from multiple speakers. Each waveform was sampled at 16 kHz and segmented into non-overlapping 15 ms frames (240 samples per frame). For each frame a 40-bin mel-filterbank power spectrum was computed and converted to the log scale. The log scale is used because raw mel power can span roughly 13 orders of magnitude; converting to dB compresses this dynamic range to about 80 dB, yielding numerical values that are easier to model and compare.

After feature extraction each utterance is represented as a variable-length sequence of shape $(n_{\text{frames}}, 40)$. From the training set we found MAX_FRAMES = 54 (used for subsequent padding/truncation); this value was computed using training data only to avoid test leakage.

4.2 Part 1 — Baseline: Average Frame Method

The baseline representation collapses an utterance into a single fixed-length vector by averaging all frame features across time, producing a 40-dimensional vector per utterance. This is the simplest fixed-length representation and discards all temporal dynamics. Classification was performed with an SVM using an RBF kernel (C=10, gamma="scale") trained on StandardScaler-normalized features.

The baseline SVM achieved 48.3% test accuracy. Measured classifier timings were: Train: 117.8 ms, Test: 18.8 ms. The low accuracy is explained by the representation choice: averaging removes temporal order and thereby destroys information that distinguishes many digits. For example, digits whose discriminative cues are temporal (the order and timing of formant energy) can have very similar mean spectra even though they are perceptually different.

4.3 Part 2 — AutoEncoder Utterance Representation

4.3.1 Padding and Flattening

Utterances vary in length, so each sequence was padded with zero-frames up to MAX_FRAMES=54 or truncated if longer. After padding each utterance becomes a fixed-size matrix of shape 54×40, which was flattened into a 2160-dimensional vector. Padding was determined using the training split only and applied identically to train and test to avoid leakage.

4.3.2 AutoEncoder Architecture

An autoencoder was trained to reconstruct the 2160-dimensional input. The encoder/decoder architecture is shown in Table 4.1.

Table 4.1: AutoEncoder architecture used for Problem 4.

Layer	Size
Input	2160
Encoder FC + ReLU	512
Encoder FC + ReLU	256
Bottleneck FC + ReLU	**128**
Decoder FC + ReLU	256
Decoder FC + ReLU	512
Output FC (linear)	2160

The autoencoder was trained with mean-squared-error reconstruction loss using Adam (learning rate 1×10^{-3}), batch size 64 for 100 epochs. Input features were StandardScaler-normalized before training. The training loss fell rapidly during the first epochs (epoch 10: 0.209 $\rightarrow$ epoch 20: 0.137 $\rightarrow \ldots \rightarrow$ epoch 100: 0.039), with steep early improvement and gradual convergence toward epoch 80–90.

4.3.3 Feature Extraction and Classification

After training, the 128-dimensional bottleneck activations were used as utterance feature vectors. An SVM with RBF kernel (C=10, gamma="scale") was trained on these latent vectors using StandardScaler normalization. The AE pipeline plus SVM achieved 93.3% test accuracy. Measured classifier timings were: Train: 88.2 ms, Test: 34.0 ms. Note that AE training (several minutes on CPU) is a one-time preprocessing cost and is excluded from these classifier timings.

4.4 Results

4.4.1 Summary Table

The quantitative results are summarised in Table 4.2.

Table 4.2: Problem 4 — Method comparison

Method	Accuracy	Train Time	Test Time
Baseline (Average Frame)	48.3%	117.8 ms	18.8 ms
AE (Pad+Encode+SVM)	**93.3%**	88.2 ms	34.0 ms

Note: AE training time (several minutes on CPU) is a one-time preprocessing cost and is not included in the classifier timing numbers above.

4.4.2 Figure 1 — Confusion Matrices

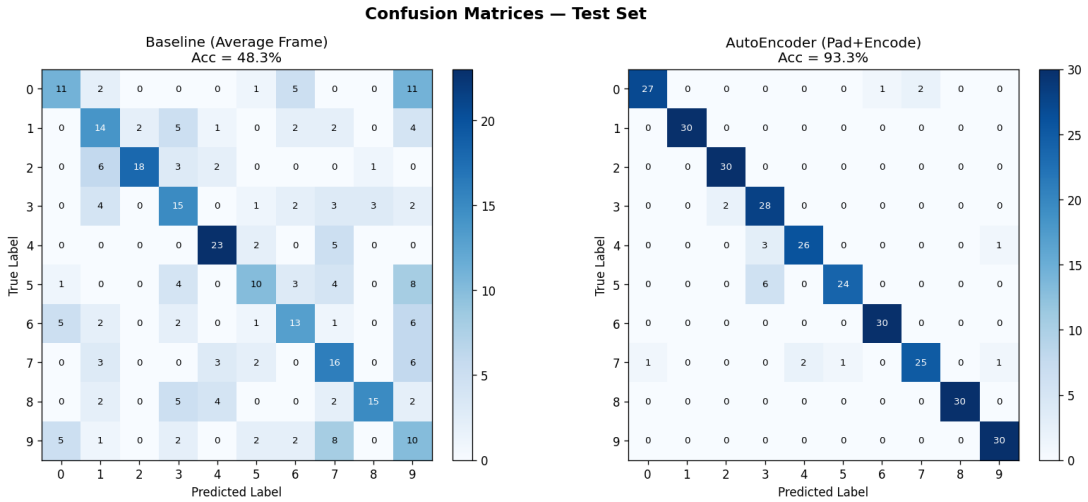


Figure 4.1: Confusion matrices for the Baseline (left) and AE (right) systems.

The baseline confusion matrix (left panel of Figure 4.1) shows widely scattered predictions with only weak diagonal structure; many digits are confused with acoustically similar alternatives (for example, 5 and 9, 0 and 9). The AE confusion matrix (right panel) is nearly diagonal: most digits are classified correctly (25–30/30). Remaining errors are isolated: digit 5 has the most confusions (6 errors, mostly with digit 3), digit 7 has 5 errors, digit 4 has 4 errors, and digits 0 and 3 each have 3 errors (notably digit 0 is mislabelled as 6 and 7). These correspond to acoustically hard confusions.

4.4.3 Figure 2 — Per-Digit Accuracy

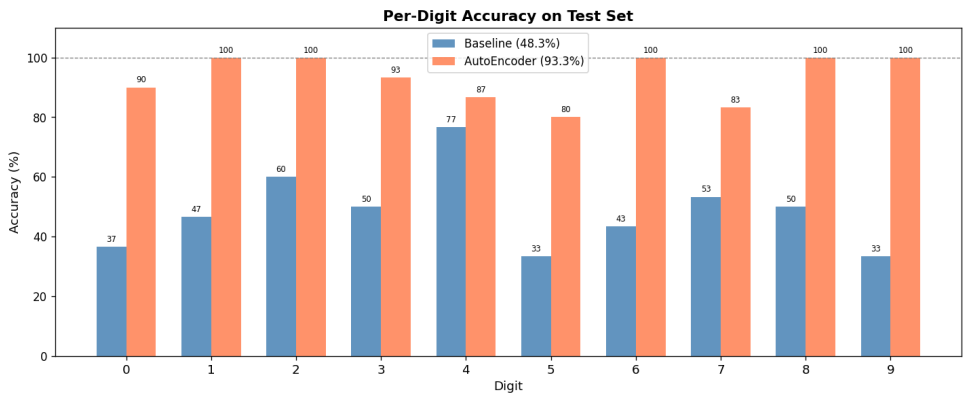


Figure 4.2: Per-digit accuracy for Baseline and AE systems.

Figure 4.2 shows per-digit accuracy. Under the baseline all digits fall between approximately 33% and 77% accuracy, with digits 5 and 9 worst (33%) and digit 4 best (77%). After AE encoding, digits 1, 2, 6, 8, and 9 reach 100% accuracy; digits 0, 3, 4, and 7 lie in the 83–93% range; digit 5 is the weakest at 80%. The AE therefore raises the floor substantially — the weakest class jumps from 33% to 80%.

4.5 Discussion

Averaging frames fails because it discards temporal structure: the mean spectrum does not encode the order or timing of spectral events that distinguish many digit utterances. Consequently the baseline accuracy is low (48.3%).

The autoencoder succeeds because by flattening a padded time sequence the encoder observes the full temporal trajectory and learns to compress discriminative temporal patterns into the 128-dimensional bottleneck. The resulting latent vectors are dense, discriminative, and compact, enabling an SVM to reach 93.3% accuracy.

Timing-wise the SVM classifiers have similar train and test times (baseline 118/19 ms, AE 88/34 ms). The AE's training cost is a one-time expense that is amortised across inference calls; at test time the cost is an encoder forward pass plus an SVM prediction, which is negligible in most deployment scenarios.

Remaining errors concentrate on acoustically ambiguous digits (notably digit 5), consistent with confusions observed in Problem 3. Compared to the Problem 3 CNN baseline (95.3%), the AE+SVM pipeline (93.3%) attains comparable performance while being computationally lighter and simpler to run: no image rendering or 2D convolutions are required.

5 Problem 5 Data Augmentation

5.1 Pipeline Overview

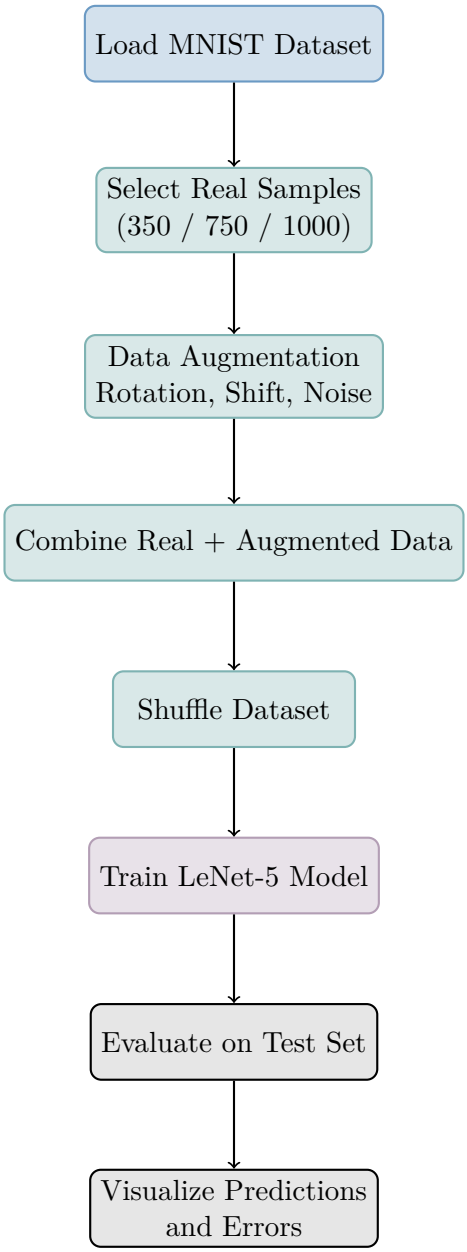


Figure 5.1: Colored pipeline of the MNIST classification system

5.2 Data Preparation

The MNIST dataset is first loaded and normalized to the range [0,1]. To study the effect of dataset size, a reduced dataset is created by selecting a fixed number of samples per digit (350, 750, or 1000).

This ensures balanced class distribution and allows controlled experimentation.

5.3 Data Augmentation

To improve generalization, artificial samples are generated using:

- Rotation (±5°, ±10°)
- Translation (small shifts in x and y)
- Noise addition

Figure 5.2 shows examples of generated images.

Figure 5.2: Examples of augmented MNIST digits

5.4 Model Training

A LeNet-5 convolutional neural network is used for classification. The model is trained using different combinations of real and augmented data.

Training is performed for 30 epochs with batch size 64. The training accuracy quickly reaches 100%, indicating that the model can fully fit the training data.

5.5 Training Progress

Figure 5.3 shows the learning behavior of the model. The loss decreases steadily while accuracy increases, demonstrating proper convergence.

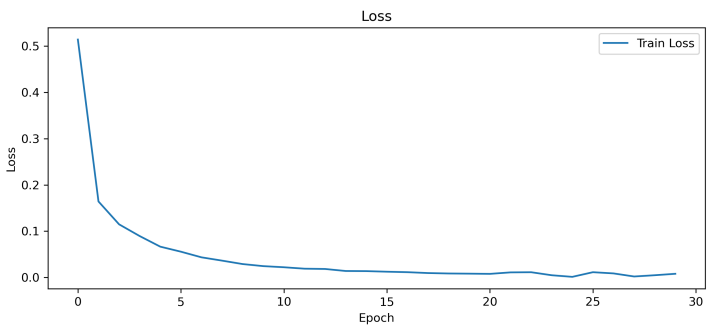


Figure 5.3: Training loss and accuracy curves

5.6 Model Predictions

Figure 5.4 shows sample predictions on the test dataset. Correct predictions are shown in green, while incorrect ones are shown in red.

The model achieves high confidence in most predictions.

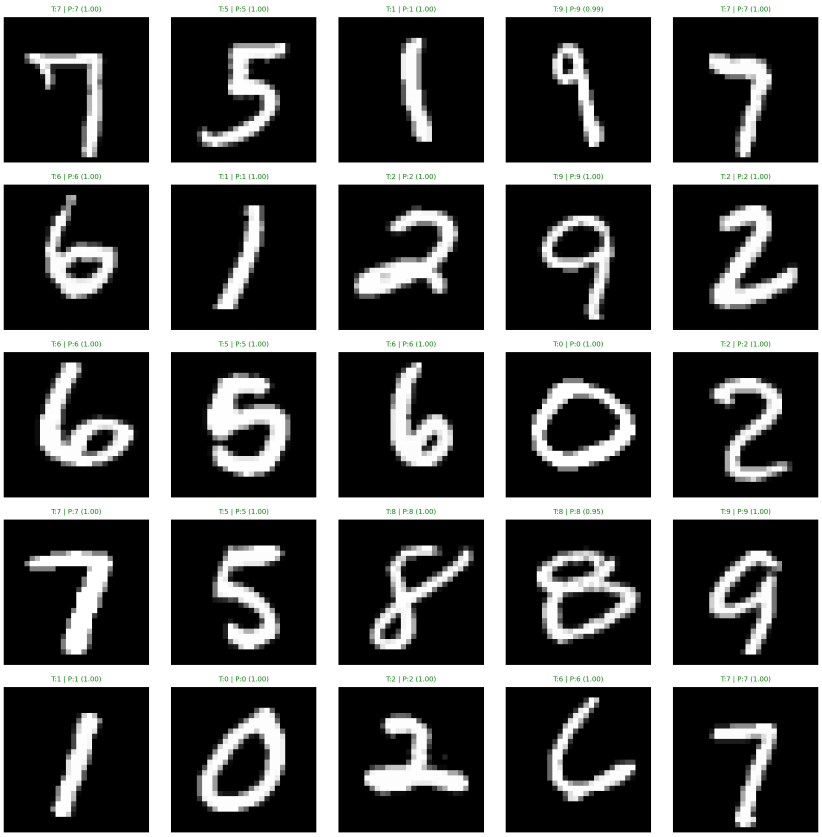


Figure 5.4: Model predictions on test samples

5.7 Error Analysis

Figure 5.5 shows misclassified samples. Most errors occur in visually ambiguous digits such as 3, 5, and 8.

This indicates that the model struggles with digits that have similar shapes.

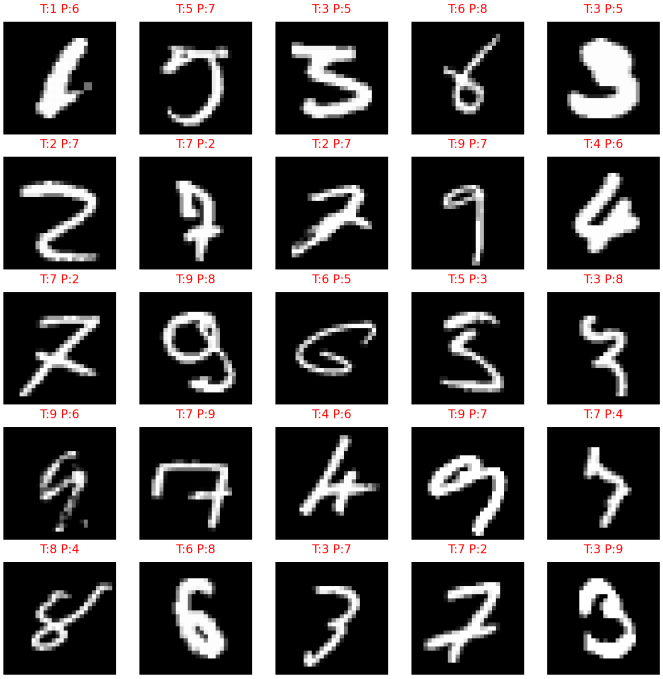


Figure 5.5: Misclassified test samples

5.8 Results and Discussion

Figure 5.6 summarizes the performance of different configurations.

The best result achieved was:

- Accuracy = 98.01%
- Configuration: 350 real + 1000 augmented samples

This demonstrates that data augmentation significantly improves performance when the amount of real data is limited.

5.9 Key Observations

- Increasing real data improves accuracy significantly.
- Data augmentation helps most when real data is limited.
- Excessive augmentation leads to diminishing returns.
- The model achieves perfect training accuracy, indicating slight overfitting.
- Test accuracy remains high, showing good generalization.

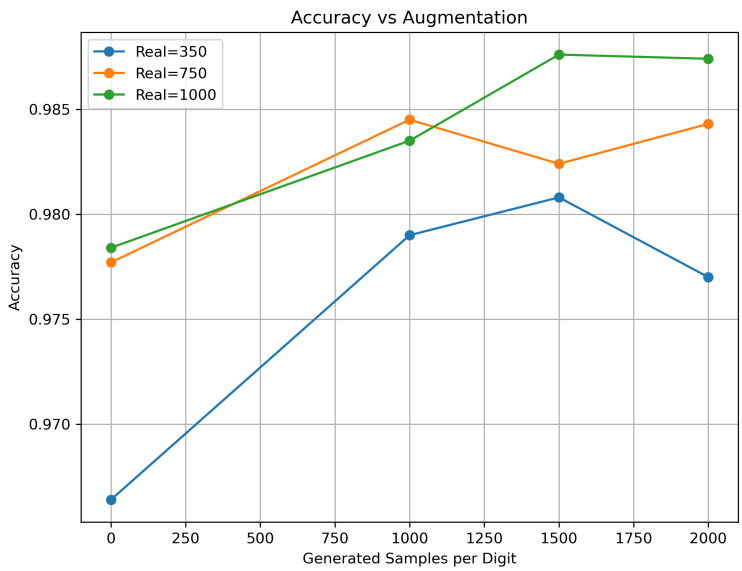


Figure 5.6: Accuracy vs number of augmented samples

6 Problem 6: GAN-based Synthetic Data Generation

6.1 Step 1: GAN Training and Initial Generation

A Generative Adversarial Network (GAN) was trained using 350 real samples per digit. The training process was monitored visually and through loss curves to ensure convergence.

6.1.1 Training Progress Visualization

As seen in Figure 6.2, the GAN consisted of a convolutional discriminator and a transposed convolutional generator. The training process was conducted for several epochs until visually acceptable digit-like samples were obtained.

The generated images were slightly blurred but still recognizable, indicating that the GAN successfully captured the general structure of handwritten digits.

Shown in Figure 6.1, from the intermediate outputs, it can be observed that the GAN gradually learns the structure of handwritten digits. Early outputs were noisy, while later epochs produced clearer and more recognizable digits.

6.1.2 Final Generated Samples

Although the generated digits are slightly blurred, they are still recognizable, indicating that the GAN successfully captured the underlying data distribution.

6.1.3 Training Stability (Loss Curves)

The loss curves show the adversarial behavior between the generator and discriminator. After initial instability, both losses stabilize, indicating that the GAN reached a relatively balanced training state.

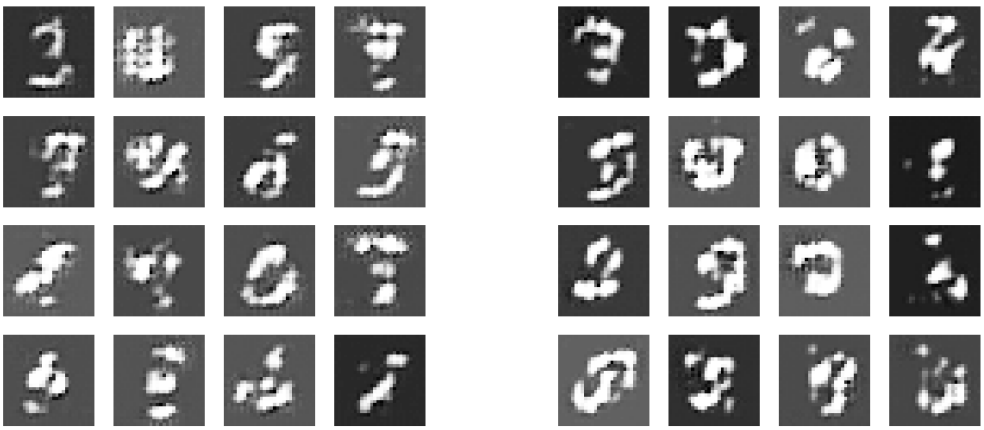


Figure 6.1: GAN generated samples during training (epochs 800 and 900)

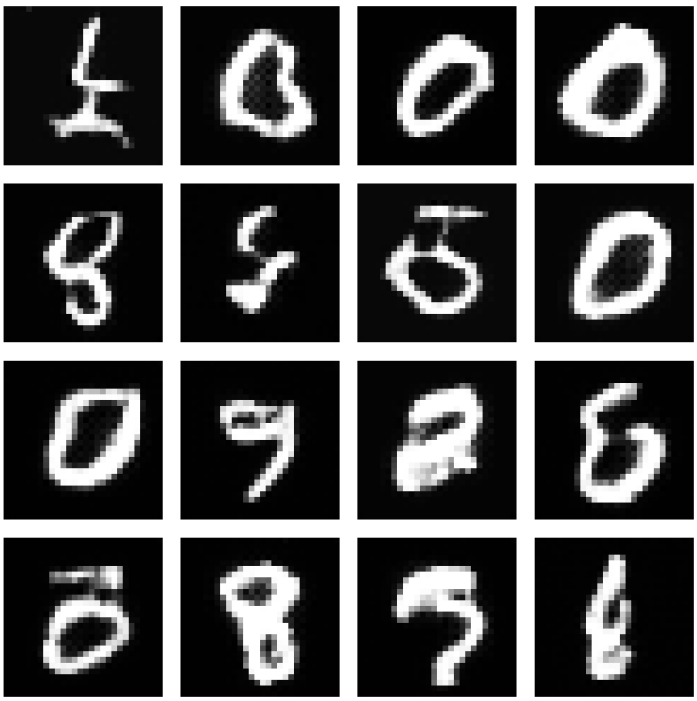


Figure 6.2: Final samples generated by the trained GAN

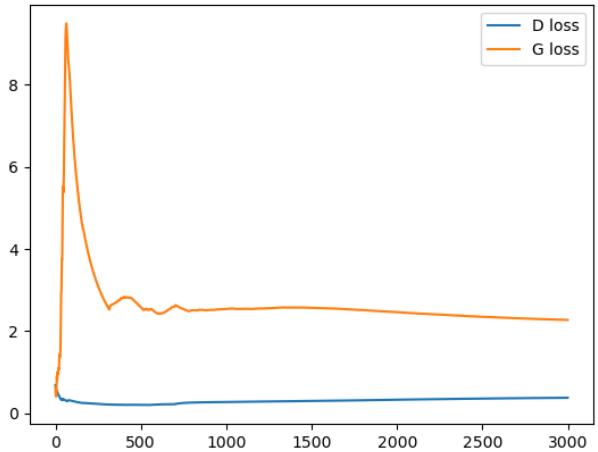


Figure 6.3: Generator and Discriminator loss during training

6.1.4 Discriminator Analysis

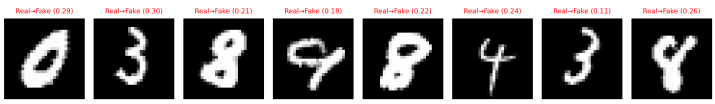


Figure 6.4: Examples where the discriminator misclassified real and fake images

Some generated samples were realistic enough to fool the discriminator, while some real images were incorrectly classified. This indicates that the generator learned meaningful patterns, but imperfections still exist.

6.2 Step 2: Effect of Synthetic Data on Classification Accuracy

The generated data was combined with different amounts of real data (350, 750, and 1000 samples per digit). The classification accuracy for each configuration is summarized in Table 6.1 and visualized in Figure 6.5.

From Table 6.1 and Figure 6.5, the following observations can be made:

- Adding synthetic data significantly improves performance when the real dataset is small (350 samples).
- The best improvement is achieved around 1000 generated samples per digit.
- Increasing synthetic data beyond this point does not improve accuracy and may slightly degrade performance.

Table 6.1: Accuracy for different combinations of real and GAN-generated data

Real per digit	**GAN per digit**	**Accuracy**
350	0	0.9548
350	1000	0.9676
350	1500	0.9651
350	2000	0.9573
750	0	0.9733
750	1000	0.9719
750	1500	0.9752
750	2000	0.9638
1000	0	0.9690
1000	1000	0.9775
1000	1500	0.9735
1000	2000	0.9740

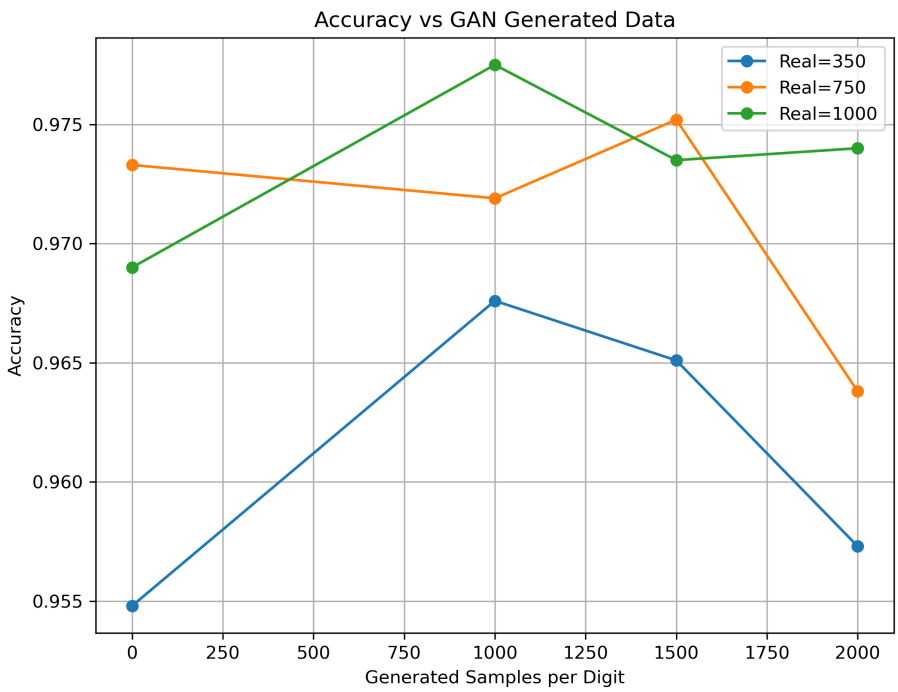


Figure 6.5: Accuracy vs. GAN-generated samples per digit

- This behavior is due to the limited quality and diversity of GAN-generated samples.

6.3 Step 3: Reducing the Need for Real Data

The baseline case using only real data (1000 samples per digit) achieved an accuracy of approximately 96.9%.

Using only 350 real samples combined with 1000 GAN-generated samples achieved an accuracy of 96.76%, which is very close to the full dataset performance.

This demonstrates that GAN-generated data can significantly reduce the need for real labeled data while maintaining high accuracy.

However, increasing synthetic data beyond an optimal point leads to performance degradation due to limited diversity and artifacts in generated images.

6.4 Discussion

The results confirm that GANs can reduce the dependency on real data while maintaining high accuracy.

- GAN-generated data is highly effective when real data is limited.
- There exists an optimal balance between real and synthetic data.
- Excessive synthetic data can reduce performance.